

Alset IoT

“Hug The Lanes”



**Octavio Morales, Allan Joseph, Anne-Marie Akoh,
Joseph Bonetti**

The future is to use IoT to make cars safer, less costly, and more
efficient!

Table of Contents

Section 1 - Introduction	4
Introduction	4
Additional Information	5
Qualifications	6
Section 2 - Functional and System Architecture	7
2.1 Sensory Input	7
2.1.1 Radio Wave Sensors	8
2.1.2 Sonar Sensors	8
2.1.3 Three-Dimensional Scanning Cameras	8
2.1.4 GPS Navigation Features	9
2.1.5 Sound Sensors	9
2.2 Linked Components	9
2.2.1 Sensor Fusion	9
2.2.2 Planning and Vehicle Control	10
2.2.3 System Management	10
Section 3 - Functional Requirements	12
3.1 Precondition and Postcondition Functional Requirements	12
3.1.1 IoT	12
3.1.2 In Motion	12
3.1.3 Not In Motion	12
3.1.4 Steering	12
3.1.5 Braking	12
3.1.6 Obstacle Detection	13
3.1.7 Screen Display	13
3.1.8 Crash Detection	13

3.1.9 Cruise Control	13
3.1.10 Technician Interface	14
3.1.10.1 Login Credentials	14
3.1.10.2 Screen	14
3.1.11 Headlights	14
3.1.11.1 Automatic Activation	14
3.1.11.2 Manual Override	14
3.1.12 Acceleration	15
3.1.12.1 Automatic Activation	15
3.1.12.2 Manual Override	15
3.1.13 Traffic Sign Detection	15
3.1.14 Sound Detection	16
3.2 Non-Functional Requirements	16
3.2.1 Security	16
3.2.2 Safety	16
3.2.3 Maintenance and Reliability	16
3.2.4 Accessibility and Comfort	16
Section 4 - Requirement Modeling	17
4.1 Use Case Scenarios	17
4.1.1 Use Case 1: Sensors detect an obstacle	17
4.1.2 Use Case 2: Technician interface	18
4.1.3 Use Case 3: Headlight activation	19
4.1.4 Use Case 4: Vehicle acceleration	20
4.1.5 Use Case 5: Sound detection	21
4.2 Activity Diagrams	23
4.2.1 Activity 1 Diagram - Obstacle Detection	23

4.2.2 Activity 2 Diagram - Technician Interface	24
4.2.3 Activity 3 Diagram - Headlight Activation	25
4.2.4 Activity 4 Diagram - Vehicle Acceleration	26
4.2.5 Activity 5 Diagram - Sound Detection	27
4.3 Sequence Diagrams	28
4.3.1 Sequence 1 Diagram - Obstacle Detection	28
4.3.2 Sequence 2 Diagram - Technician Interface	29
4.3.3 Sequence 3 Diagram - Headlight Activation	29
4.3.4 Sequence 4 Diagram - Vehicle Acceleration	30
4.3.5 Sequence 5 Diagram - Sound Detection	30
4.4 Classes	30
4.4.1 Class 1 - Obstacle Detection	30
4.4.2 Class 2 - Technician Interface	31
4.4.3 Class 3 - Headlight Activation	31
4.4.4 Class 4 - Vehicle Acceleration	32
4.4.5 Class 5 - Sound Detection	32
4.5 State Diagrams	32
4.5.1 State Diagram 1 - Obstacle Detection	32
4.5.2 State Diagram 2 - Technician Interface	32
4.5.3 State Diagram 3 - Headlight Activation	33
4.5.4 State Diagram 4 - Vehicle Acceleration	33
4.5.5 State Diagram 5 - Sound Detection	33
Section 5 - Design	4
5.1 Software Architecture	4
5.1.1 Data-Centered Architecture	number
5.1.2 Data Flow Architecture	number

5.1.3 Call Return Architecture	number
5.1.4 Object-Oriented Architecture	number
5.1.5 Layered Architecture	number
5.1.6 Model View Controller Architecture	number
5.1.7 Finite State Machine Architecture	number
5.2 Interface Design	5
5.3 Component Design	6

Section 1 - Introduction:

Opening Introduction

The 21st century has seen exponential growth in technological advancements, with the past 23 years celebrating achievements from the earliest smartphone models of contemporary tech giants to machine-learning techniques used in AI training. The overarching goal appears to be streamlining human life by reducing manual effort. For instance, smart home devices can turn on lights with a voice command, tablets and touchscreen laptops offer endless writing space, and self-driving vehicles replace both manual and automatic driving.

The future in store for self-driving cars can be greatly beneficial for modern-day and prospective drivers. More accidents can be prevented with the help of a car's algorithm that is aware of all its surroundings all the time. Those who have trouble driving or need accommodations to do so may have an easier time traveling in their own vehicles. Of course, a self-driving vehicle does not totally replace a human driver who must still take responsibility when driving. Still, as we have seen with other emerging technologies, it could make transport easier.

Using the Unified Process method, our team hopes to manufacture high-level software that can simulate that of a self-driving car. We will require both functional and non-functional components to construct our algorithms. Some features that we aim to include are: autonomous navigation and control, obstacle detection and avoidance, real-time traffic analysis and route planning and more. For the first release, for example, we plan to offer a first prototype of sensor detection. This will alert the motor vehicle about other objects in close proximity to the point of a potential collision or danger. After this first release, revisions will be performed and edits will be

made to improve upon our work. Then, subsequent releases will build upon their prior ones and follow suit.

This project is one we take seriously and maturely because there is no room for error. When developing software for items as potentially dangerous as a self-driving automobile, any incorrect bit of code can have fatal consequences. Therefore, the performance and reliability of our final product must most accurately reflect our dedication to this project. Fortunately, we can most sensibly accomplish this by following a well-structured model for developing our software. We aim to adhere to the Unified Process as our role model. In this way, we can make initial plans, gather necessary materials and resources, revise our plans if necessary, and then continue to execute them in a well controlled environment. This will also prevent us from forgetting to implement certain features or implementing ones that we eventually decide against.

This team's members have had the opportunity to gain experience in different departments and subsets of computer science and cybersecurity. Our computational skills range from basic operations on simple, linear data structures to a deeper understanding of computational theory and its representations. Furthermore, we collectively possess an arsenal of skills and qualities, including being task-oriented, patient and analytical, that will help us persevere in our project. In all, we are prepared to take on the challenge of developing this software, and we will not be satisfied until the final product best reflects our tenacity.

Additional Information

- 1. Goal: Developing a systematic infrastructure using software and hardware to create a car with autonomous navigation and control**
- 2. Datasets**
 - a. Hardware**
 - i. GPS data (positioning and navigation)
 - ii. Camera data (detecting road signs, traffic lights, and obstacles)
 - iii. Ultrasonic sensors (measuring distance from nearby objects)
 - iv. Radars for (detecting other vehicles and objects in the road)
 - b. Software**
 - i. Information from the driver's control inputs, such as steering wheel and pedals.
- 3. Use IoT to make cars safer, less costly, and more efficient**
 - a. Use IoT to make decisions or provide information to the driver
 - b. Use IoT edge devices that can capture data from the car and environment
 - c. Use an analytic engine (IoT Engine) on board to process information and make decision that is passed on to the Car Control System (CCS)
 - d. Provide capability for the driver to enter commands and receive status

- e. Download the latest rules and updates for operation from the Fog/Cloud into the IoT Engine
- f. Upload the car data to the company cloud

Team Roles and Qualifications:

Leader: Anne-Marie Akoh

Members: Joey Bonetti, Allan Joseph, Octavio Morales

<u>Name: Anne-Marie Akoh</u> <u>CS Courses Taken:</u> CS 115, CS 146, CS 284, CS 135, CS 382, CS 385, CS 334, CS 306 <u>Languages:</u> Python, Java, Javascript, HTML, CSS, C++, Assembly <u>Qualifications:</u> Writing, Patience, Task-Oriented	<u>Name: Joey Bonetti</u> <u>CS Courses Taken:</u> CS 115, CS 146, CS 284, CS 135, CS 382, CS 385, CS 306 <u>Languages:</u> Python, Java, Javascript, HTML, CSS, C++, Assembly <u>Qualifications:</u> Analysis, Curiosity, Task-Oriented
<u>Name: Allan Joseph</u> <u>CS Courses Taken:</u> CS 115, CS 101, CS 284, CS 135, CS 382, CS 385 <u>Languages:</u> Python, Java, Assembly, Swift, C, C++ <u>Qualifications:</u> Analysis, Curiosity, Coding	<u>Name: Octavio Morales</u> <u>CS Courses Taken:</u> CS 115, CS 101, CS 284, CS 135, CS 382, CS 385 <u>Languages:</u> C#, Python, Java, C, C++, Assembly <u>Qualifications:</u> Coding, Outgoing, Patience

Section 2 - Architecture:

Functional and System Architecture:

We shall now detail the functionality of the architecture we plan to implement for our self-driving car. It is incredibly important that the architectural interfaces we develop connect seamlessly with one another in order to ensure a fluid and unbroken highway of data transmission and communication between components. Importantly, we look to involve the following sensory modules; radio wave detectors, sonar detectors, three-dimensional scanning cameras, and GPS navigation features. Ideally, we need to ensure that each part works flawlessly for the self-driving car to be used publicly. As shown in *Figure 2.1*, a part will more than likely have a direct impact on the performance of hardware used later, so the failure of one part can result in the failure of the intentional self-driving car functionalities. When the architecture is implemented, it will result in many of the features for the self-driving car.

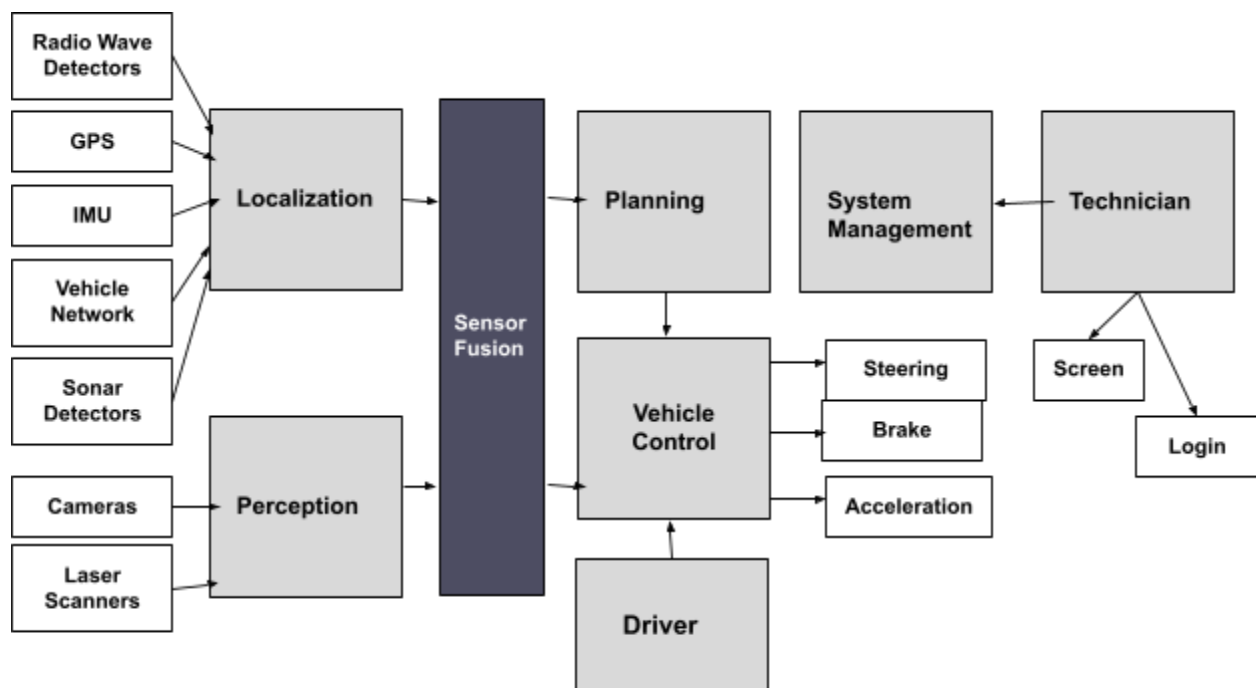


Figure 2.1

2.1 Sensory Input:

The following sensor descriptions will describe the different types of sensory input the vehicle expects to receive. The data provided by each will be fed to the different components which will then make safe decisions about what to do next.

2.1.1 Radio Wave Sensors

The inclusion of sensory detectors of radio waves on the electromagnetic spectrum will be useful in providing information about an object's position in relation to items around it. In terms of a self-driving car, these sensory detectors can pick up radio wavelengths of varying frequencies depending on the proximity of a surrounding object. The closer the object is to the car, the more wary our sensors will be until it reaches a point of possible collision. The detectors will track the distance between the car and other nearby objects, record the angles of closeness between them and measure, if applicable, the speed at which they are traveling. In this way, necessary information will be provided to the car's other functional components so that they can further build off it.

2.1.2 Sonar Sensors

As important as it is to have radio wave detection of physical objects, it is to have sonic detection of our surroundings. With the help of the sonar functional capabilities we shall implement, our audible detecting devices will be able to determine the approximate closeness of an based on how soon sound waves are transmitted back. Emitting sound waves from the architectural system and tracking how long it takes to receive their echoing counterparts back will work hand-in-hand with our radio wave detectors. These two sensory subsets will provide a combined effort of judging the position of the car in relation to potential hazards around it. To ensure an extra safety check, these sonar detectors will give and receive information to and from the radio wave detectors. Readings of the same external object's proximity should be the same if not minutely different between the two. When this is the case, we will have ensured that our car has reliable means for understanding its own positional status.

2.1.3 Three-Dimensional Scanning Cameras

While intangible methods for determining current position are incredibly useful and important, they are not necessarily user-friendly. This is essentially because a passenger may be able to derive little to nothing from radio wave and sound wave detectors alone. Humans tend to be more visual creatures and can learn a great deal simply from what they see. This is why it is important for our car to also include cameras to provide a sense of direction, position, and proximity to other objects. Our three-dimensional scanning cameras will receive information provided from both the radio wave and sonar sensors and incorporate into projecting a visual image that represents the numerical data. In this way, not only passengers but the car itself will have a visual representation of where they currently are, where they are going, and what is surrounding them. It is also possible that, in rare cases, radio wave or sonar sensory systems may be faulty. In this event, having image sensors can still inform the car of where it is in relation to others. However, these cameras can only collect visual data close enough to be in camera view. This is why it is necessary to have varied means of positional detection to account for things that may not be visually detectable. Therefore, the radio wave sensors, sonar sensors and these visual image sensors will communicate with each other to gather, read, and interpret information about

the car's position. Should one functional component fail, there will still be others to make up for it as much as possible until the necessary repairs are made.

2.1.4 GPS Navigation Features

Of course, it is necessary that a self-driving car has sensory architecture to relate its position to that of other objects or items nearby. Still, this does not and cannot act as any sort of geographical locating substitute. Essentially, the aforementioned sensors can describe where a car is in a local and relative sense, but it cannot do so in a global sense. These sensors do not know where in the world they are but do know where they are relative to objects just close enough to be detected. This is why it is important for us to also incorporate a geolocating component in our car's functional architecture. This way, our self-driving automobiles can have an understanding of where they are in the world. With the use of nationwide and worldwide satellite systems of navigation - more commonly known as Global Positioning System, or GPS - our car's geolocating sensors can read information provided by them and share it with the car's other functional components. As such, the radio wave, sonar and image-detecting sensors will interact with each other to sense the local position of the car while these geographical ones will interpret global position. This will be a great aid in directing the car to travel to certain locations using a navigation system. Not only will the car drive with full comprehension of how close it is to other objects, but it will also be able to travel to specific geographic coordinates based on the GPS satellite information it receives on the way there.

2.1.5 Sound Sensors

Sound sensors are a crucial component in autonomous cars because they help detect and interpret auditory cues from the surrounding environment. Sound sensors use microphones and audio signal processing techniques to identify specific sounds on the road, such as horns, sirens, and other emergency vehicle signals, and feed this information into the car's control system. The ability to detect these sounds is important for several reasons. Firstly, it can provide the car with information about its immediate environment. For instance, a sudden horn blast can indicate a potential danger or obstacle on the road, prompting the car's control system to adjust the speed and direction of the vehicle accordingly. Similarly, the detection of emergency vehicle sirens can allow the car to give way and make room for the approaching vehicle. Additionally, sound sensors can help improve safety by providing the car with information about the behavior of other drivers on the road. For example, the detection of a car horn or emergency vehicle siren can indicate that other drivers are trying to alert the autonomous car of their presence or intentions.

2.2 Linked Components:

Now, the linked components we shall describe will explain how the aforementioned sensory input will influence the communication between interfaces in the car's architecture.

2.2.1 Sensor Fusion

The cooperation and collaboration of different sensors embedded in the car's architecture will equip the vehicle with enough information to make informed and intelligent decisions about its movement. This sensor fusion is necessarily made up of both localization and perception, each with their own subsets of components to build them. We analyze each individually.

Localization:

Sensors can be used to detect and analyze the environment around the vehicle and provide real-time feedback to the driver. For example, cameras and lidar sensors can detect other vehicles, pedestrians, and obstacles on the road, while GPS sensors can provide location information. By integrating these sensors into the driver assistance system, the architecture can be validated for its ability to provide accurate and reliable information to the driver, thus ensuring their safety.

Perception:

To ensure that the moving vehicle has a good and accurate understanding of its current position and surroundings at any point, we will rely on the information relayed by perception-based sensors. Both cameras and laser scanners will receive environmental input from around the car, convert it into information readable by the vehicle, and finally send that data to the other components of the architecture. This way, decisions of movement that require a current understanding of the car's position can be well judged and safely made.

2.2.2 Planning and Vehicle Control

The critical information received from the sensor fusion component in this vehicle's architecture will be necessary in planning and controlling movement of the car. The combined data provided by both the localization and perceptions fields of sensors is required in order for the self-driving car to decide where to move next, when it should do so, and at which speed to do so. These planned decisions will feed into the vehicle control component which will physically carry out those plans. For example, the degree by which to turn the steering wheel, the speed at which to hold down the brake pedal, and the duration for which the car should be accelerating are all possible factors that may be directed by controlling the vehicle.

However, despite the car possessing self-driving capabilities, it is still necessary to allow for user input - that is driver input. The driver of the car must also be able to influence vehicle control. This acts as a safety measure for when sensory information is either miscommunicated in transmission or incorrectly received in the first place. Similarly, there may be times when the driver makes unintended or potentially dangerous decisions. In which case, the correctly received sensory input will prevent the driver from carrying out those decisions. The two keep each other in check in this way.

2.2.3 System Management

In self-driving vehicles, sensors play a critical role in making decisions and taking actions. For instance, radar sensors can detect the distance and speed of surrounding vehicles,

while ultrasonic sensors can detect the proximity of obstacles. By using machine learning algorithms to process the data from these sensors, the self-driving system can make informed decisions and take necessary actions to ensure safety. Any changes made to the architecture must take into account the impact on the accuracy and reliability of the sensors and their ability to make decisions. Therefore, it is important to trust that the system of the vehicle is always as up to date as possible. A qualified and professional technician will be the recipient of the current status of the car's performance. Should there be any issues or potential hazards, the technician will revise the car's architectural components, identify the issues, and reconstruct faulty parts if necessary. Doing this regularly will ensure that the self-driving vehicle is well maintained and constantly safe to travel in.

System Admin

In a system administration setting, sensors can be used to monitor the health and performance of the network and cloud infrastructure. For example, temperature and humidity sensors can monitor the server room, while network sensors can monitor network traffic and identify potential security threats. By using this data to analyze the performance of the network and cloud connection, system administrators can run diagnostic evaluations to validate the architecture, detect issues or potential crises, and make informed decisions about making any drastic changes.

Section 3 - Functional Requirements

3.1 Precondition and Postcondition Functional Requirements:

3.1.1 Iot:

Precondition: Car is on.

Postcondition: The car's internal connections are established using IoT.

Functions In actions: All systems and sensors are connected and active

3.1.2 In Motion:

Precondition: Battery is charged and all components and sensors are functioning properly.

Postcondition: The car safely completed its journey without issue. And battery charge levels are checked for future trips.

Functions In actions: Battery charge level sensor, Motor sensor, Brake sensor, GPS sensor, Tire pressure sensor

3.1.3 Not In Motion:

Precondition: Car is parked, battery is charged, and doors are locked.

Postcondition: The battery remains sufficiently charged after diminution in charge levels after idling-by, or battery is charging. And doors remain locked.

Functions In actions: Battery charge level sensor, Camera system

3.1.4 Steering:

Precondition: Car sensors and GPS function properly, obstacles in the path.

Postcondition: The car safely completed the turn and wheels are returned to initial position.

Functions In actions: GPS Navigation, Radio Wave Sensor

3.1.5 Braking:

Precondition: Car sensors detect an obstacle or danger, or the driver initiates the braking process.

Postcondition: The car has come to a complete stop, and obstacles or danger has been avoided.

Functions In actions: Sonar sensor, Camera system, Three-Dimensional Scanning Cameras

3.1.6 Obstacle detection:

Precondition: The sensors are calibrated and are providing accurate distance and location data

Postcondition: The car has initiated an appropriate response, such as warning the driver, slowing down, or automatically applying the brakes to avoid the obstacle.

Functions In actions: Sonar sensor, Radio Wave Sensor, Camera system

3.1.7 Screen Display:

Precondition: Car's display is powered on and functioning properly.

Postcondition: The display shows all necessary information for the driver, such as speed, range, and other critical details, and remains visible until the car is powered off.

Functions In actions: Screen

3.1.8 Crash Detection:

Precondition: Impact detected

Postcondition: The emergency services are notified and the car shuts down.

Functions In actions: All sensors in Sensor Fusion are on standby awaiting any changes.

3.1.9 Cruise Control:

Precondition: Permission given by driver

Postcondition: The vehicle will allow its self-driving capabilities to monopolize the entire system.

Functions In actions: All sensors in Sensor Fusion take complete control of the automobile and communicate with each other through their own component interfaces.

3.1.10 Technician Interface:

3.1.10.1 Login Credentials:

Precondition: System Management software prompts the technician to enter identifiable credentials to log into the system.

Postcondition: The technician will log in using their personal and private credentials to perform diagnostic assessments, update the current software or retrieve log files.

Functions In actions: The System Management component

3.1.10.2 Screen:

Precondition: The vehicle's built-in screen will require software updates to be sent to it.

Postcondition: The technician will have access to the car's screen display so that they may communicate software updates to it and resolve any bugs.

Functions In actions: The car's Screen

3.1.11 Headlights:

3.1.11.1 Automatic Activation

Precondition: Light sensors in Sensor Fusion will detect the brightness of the outside environment.

Postcondition: The vehicle's headlights will, after a certain threshold, automatically turn on the car's headlights.

Functions In actions: All light-detecting and external light-triggering sensors in Sensor Fusion will be engaged.

3.1.11.2 Manual Override

Precondition: Specific input from the driver in terms of whether or not the headlights are activated.

Postcondition: The vehicle's headlights will manually be turned on or off by the driver if not already on or off automatically.

Functions In actions: All light-detecting and external light-triggering sensors in Sensor Fusion will be engaged. Vehicle control also engages due to direct driver input.

3.1.12 Acceleration:

3.1.12.1 Automatic Activation

Precondition: Sensors in Sensor Fusion detect enough space ahead and around the vehicle to safely accelerate for t number of seconds.

Postcondition: The car will accelerate for t seconds.

Functions In actions: Localization and Perception sensors of Sensor Fusion will influence Vehicle Control which will enable the acceleration of the car.

3.1.12.2 Manual Override

Precondition: The driver specifically pushes on the acceleration pedal.

Postcondition: The car will accelerate for as long as the driver is pushing the pedal providing no Crash/Obstacle Detection features become turned on.

Functions In actions: The Driver influences the Vehicle Control Component to accelerate the car.

3.1.13 Traffic Sign Detection:

Precondition: Sensors are calibrated and functioning properly to detect and process the visual data in the environment.

Postcondition: The car will process the visual data collected and will respond accordingly to the traffic law that the sign implies.

Functions In actions: Three-Dimensional Scanning Cameras

3.1.14 Sound Detection:

Precondition: Sensors are calibrated and functioning properly to detect and process the visual and audio data in the environment.

Postcondition: The car will process the visual and audio data collected and will respond accordingly to what is creating the sound and the sound that is created whether it be a horn or a vehicle of priority like an ambulance or fire truck.

Functions In actions: Three-Dimensional Scanning Cameras and Sound Sensors

3.2 Non-Functional Requirements:

3.2.1 Security

- IoT shall notify owner after operator or admin fails to log in three times
- IoT shall encrypt owner PIN of the vehicle
- Only verified operators shall be able to operate the vehicle

3.2.2 Safety

- Automatic emergency braking to prevent collisions
- Crash detection and emergency services notification
- Automatic shutdown to prevent further damage
- High voltage safety systems to reduce the risk of electric shock

3.2.3 Maintenance and Reliability

- Software Updates shall be install immediately upon being push publicly
- All sensors and systems shall be operating normally being able to be driven on the road

3.2.4 Accessibility and Comfort

- Internal LED Lights can be set to a specific color by the driver or may run on a preset color-changing mode.
- Seat and steering wheel heaters will be turned on or off by the driver who will also be able to set the heat level.

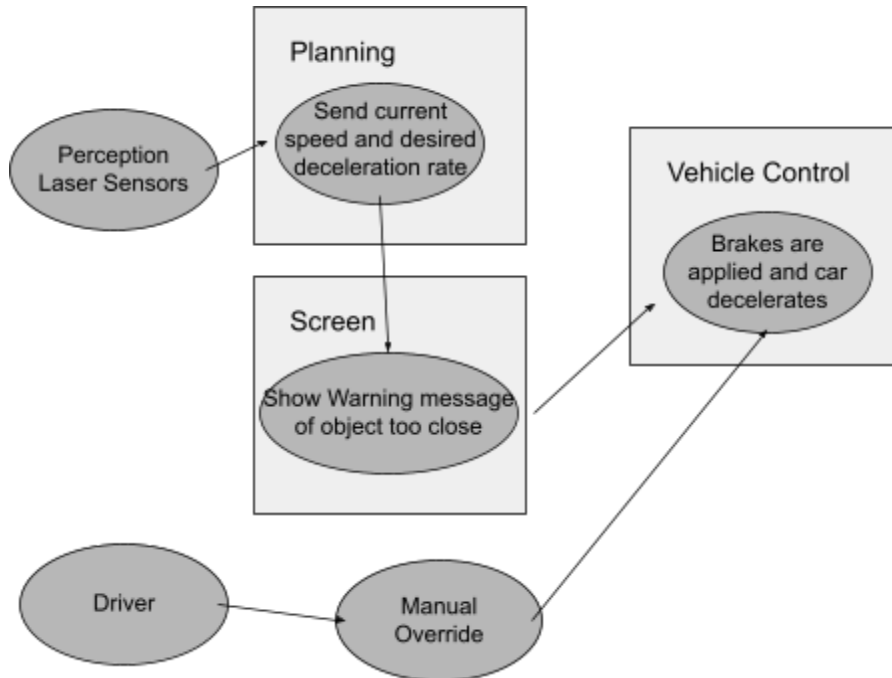
Section 4 - Requirement Modeling

4.1 Use Case Scenarios

The following five Use Cases describe the expected modeled behavior of some sample requirements. They each annotate a pre-condition, post-condition and necessary trigger to initiate them. Each is then followed by a numbered list chronologically describing the expected order of events based on the conditions and trigger. Finally a Use Case diagram offers a visual representation of this information.

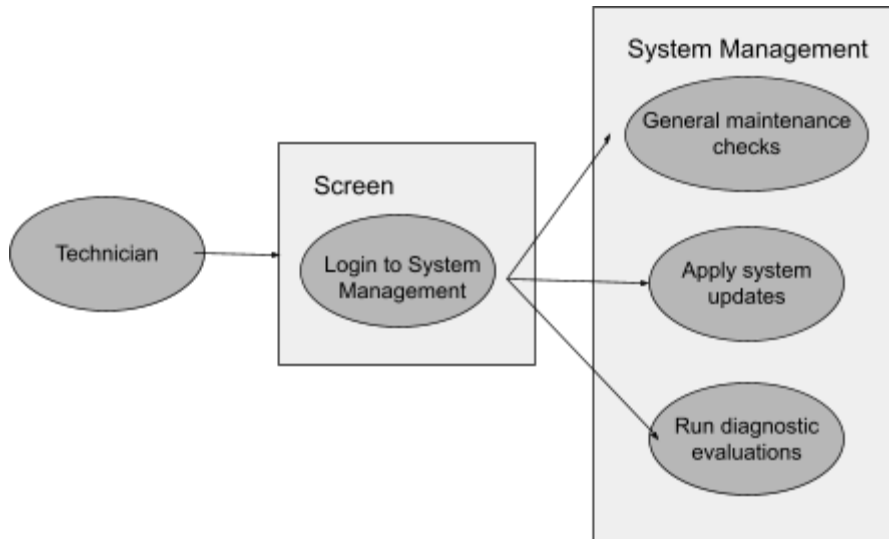
4.1.1 Use Case 1: Sensors detect an obstacle

- *Pre-condition:* The sensors are calibrated and are providing accurate distance and location data. They receive wavelength information from an object at most 500 ft away.
 - *Post-condition:* The car has initiated an appropriate warning to the driver through the screen, slowed down for as long as the object is within 500 ft.
 - *Trigger:* The sensors detect an obstacle 500 ft away or closer
1. The car is already on and in motion
 2. Perception laser sensors read wavelengths of an object at most 500 ft away from the vehicle
 3. This information is sent to Sensor Fusion
 4. Screen display shows Warning message to driver
 5. Speed Sensor sends current speed and desired deceleration rate to Planning.
 6. Planning decides to make the car decelerate at the specified rate
 7. Sensors check if object is now at least 500 ft away. If so, the object is no longer a threat. If not, deceleration continues for as long as necessary
 8. Exception: If driver manually overrides braking and applies the brake pedal themselves, the car will decelerate at the rate of the driver's own braking. Otherwise, will decelerate at rate provided by sensors.
- Use Case 1 Diagram:



4.1.2 Use Case 2: Technician interfaces with the screen software

- *Pre-condition:* The vehicle's built-in screen will require software updates to be sent to it. The technician may access this software after logging in with their private and secure credentials
 - *Post-condition:* The technician will have access to the car's screen display so that they may communicate software updates to it and resolve any bugs.
 - *Trigger:* The software needs updating
1. The car is already on and not in motion
 2. The assigned technician will be prompted by the Screen to enter their login credentials
 3. A successful login takes them to software System Management
 4. Algorithmic updates created by the technician are fed to System Management which will now incorporate them going forward
 5. Exception: If technician has remote access to the car's software, they may access it virtually to perform system diagnostics and updates.
 6. Exception: even if no system update is required, the technician may still be allowed to login to System Management for general maintenance checks
- Use Case 2 Diagram:



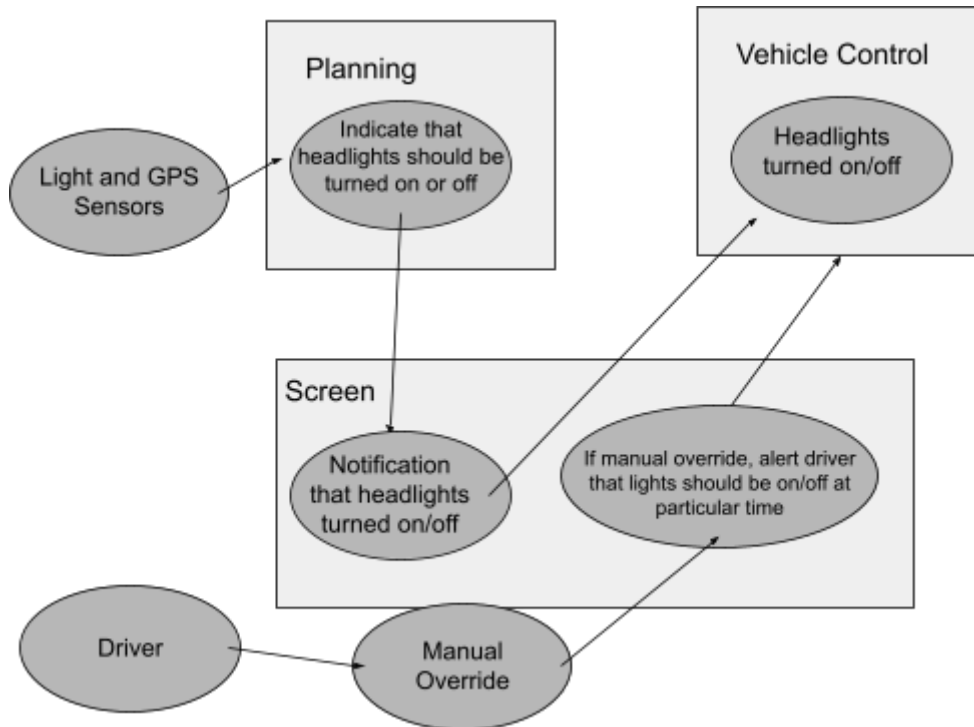
4.1.3 Use Case 3: The vehicle's headlights are activated

- *Pre-condition:* Light and GPS sensors in Sensor Fusion detect the brightness of the outside environment. Specifically, they measure 30 minutes of light change after sunset or before sunrise.
- *Post-condition:* The vehicle's headlights will, after this certain threshold, automatically turn on the car's headlights.
- *Trigger:* 30 minutes after sunset, light sensors will prompt for headlights to be turned on

1. The car is already on
2. GPS sensors detect the localization of the car and, based on this, when the sun is expected to set for the evening or rise for the morning
3. Once sunset begins, or before sunrise, Light Sensors measure the brightness of the environment for 30 minutes.
4. For after sunset: If, after this time, it has indeed gotten darker, this information is sent to Sensor Fusion. If not, Light Sensors continue detecting until darker.
5. For before sunrise: If, after this time, it has indeed gotten brighter, this information is sent to Sensor Fusion. If not, Light Sensors continue detecting until brighter.
6. Headlight Sensor indicates to Planning that headlights should be turned on or off according to the local time of day and sun position.
7. Planning decides to activate or deactivate the headlights
8. Screen display shows message to driver indicating that headlights have been turned on or off
9. Exception: If driver manually overrides headlight activation by touching a button on the Screen Display to turn them on/off, the car's headlights will do as the driver instructs.

However, a warning message will be displayed if headlights are really supposed to be on or off at that given time and the driver is trying to do otherwise.

- Use Case 3 Diagram:

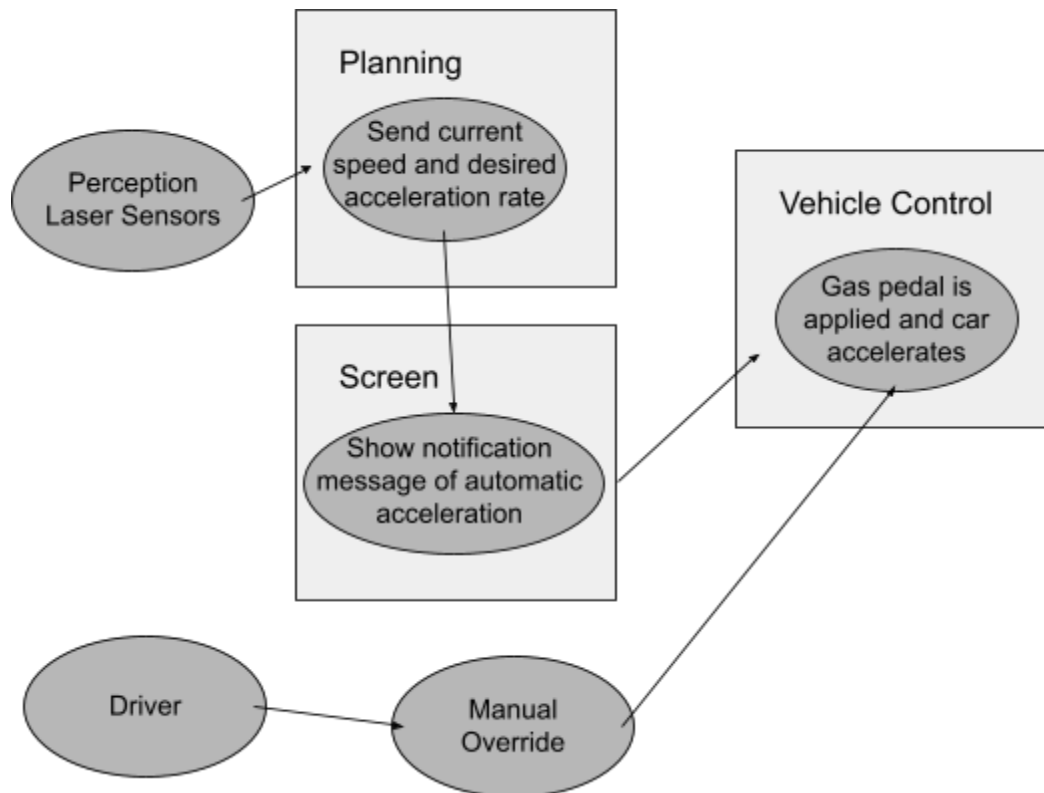


4.1.4 Use Case 4: The vehicle is accelerated

- *Pre-condition:* Perception Sensors in Sensor Fusion detect enough space ahead and around the vehicle to safely accelerate for t number of seconds.
 - *Post-condition:* The car will accelerate for t seconds.
 - *Trigger:* A safe amount of surrounding space for acceleration is detected.
1. The car is already on and in motion
 2. Perception laser sensors read wavelengths of an object at least 1500-3000 ft away from the vehicle.
 3. This information, along with the expected time to closest object at current speed, is sent to Sensor Fusion
 4. Speed Sensor sends current speed and desired acceleration rate to Planning.
 5. Planning decides to make the car accelerate at the specified rate for t number of seconds
 6. Screen display shows message to driver indicating that the car is automatically accelerating
 7. Sensors check if closest object is still 1500-3000 ft away. If so, may continue to accelerate. If not, acceleration terminates.

8. Exception: If driver manually overrides accelerating and applies the acceleration pedal themselves, the car will speed up at the rate of the driver's own acceleration. Otherwise, will speed up at rate provided by sensors for as long as is safe.

● Use Case 4 Diagram:

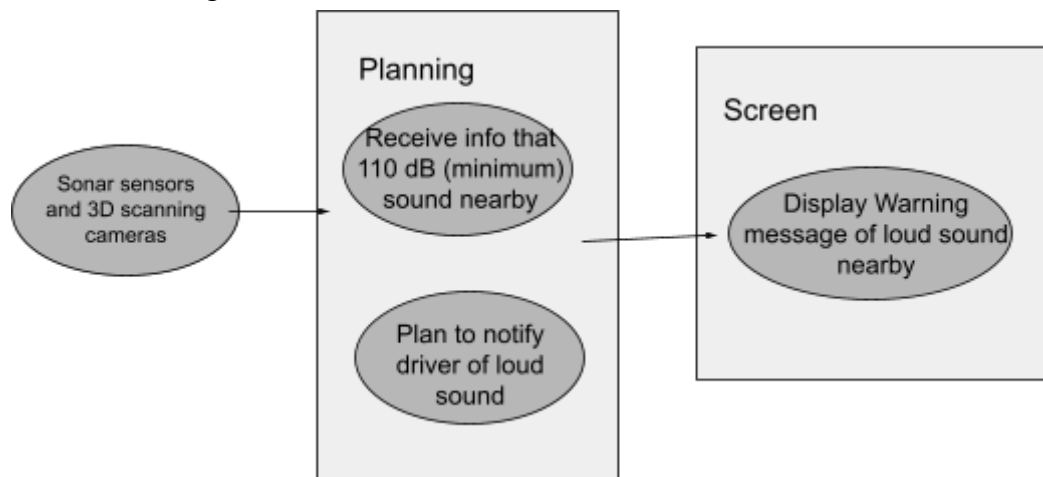


4.1.5 Use Case 5: Sensors detect a sound

- *Pre-condition:* Sensors are calibrated and functioning properly to detect and process the visual and audio data in the environment.
 - *Post-condition:* The car will process the visual and audio data collected and will respond accordingly to what is creating the sound and the sound that is created whether it be a horn or a vehicle of priority like an ambulance or fire truck.
 - *Trigger:* The sensors detect a loud sound of 110 decibels or higher
1. The car is already on and in motion
 2. 3D Scanning Cameras and Sonar Detectors are engaged and constantly reading the environment.
 3. Sonar Detectors read the wavelengths of a sound of at least 110 decibels.
 4. 3D Scanning Cameras use information from where the Sonar Detectors read the sound to visually locate the object it is likely coming from (i.e. a 110 decibel car horn coming from the car beside)

9. This information is sent to Sensor Fusion
10. Sound Sensor sends indication message to Planning to tell it that a nearby loud sound has been detected.
11. Planning decides to notify the driver of this sound
12. Screen display shows Warning message to driver of a loud sound to be weary of. This may be of a nearby car honking its horn, of an approaching ambulance to make room for, a nearby car crash to prepare to avoid etc...
13. Sensors check if sound is now fewer that 110 decibels loud. If so, the sound is no longer a concern. If not, Warning message continues to be displayed for as long as necessary.

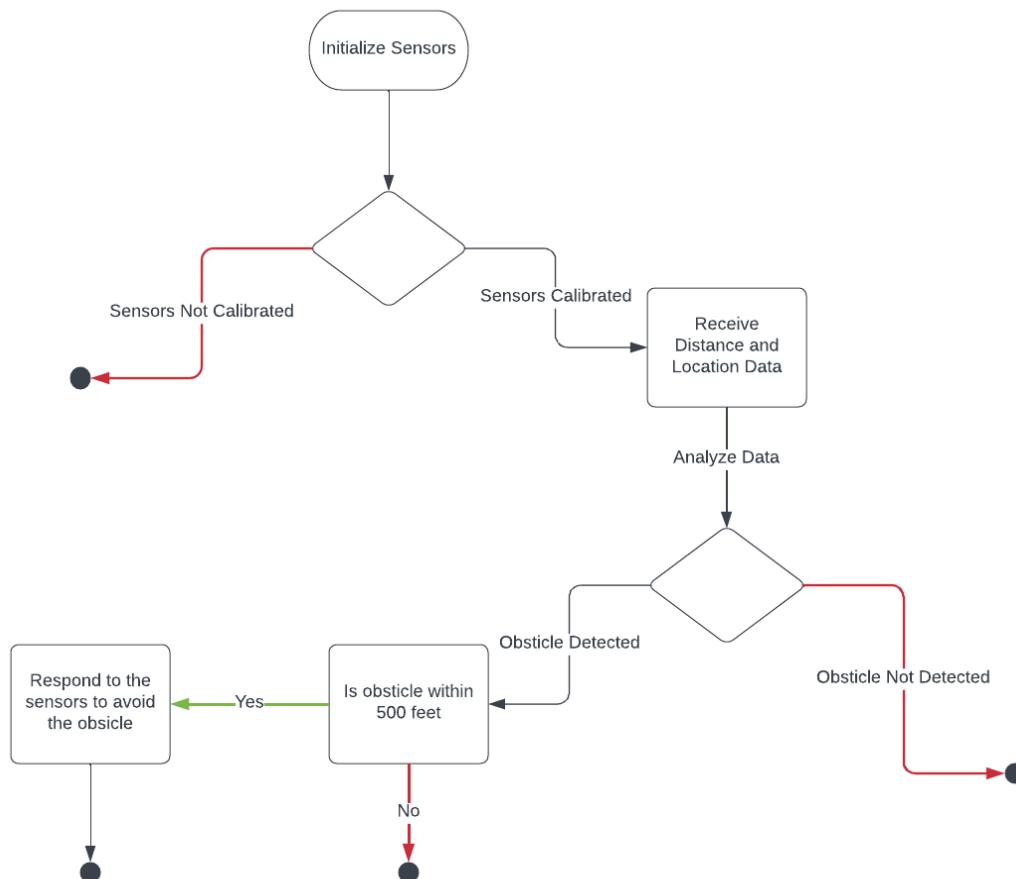
- Use Case 5 Diagram:



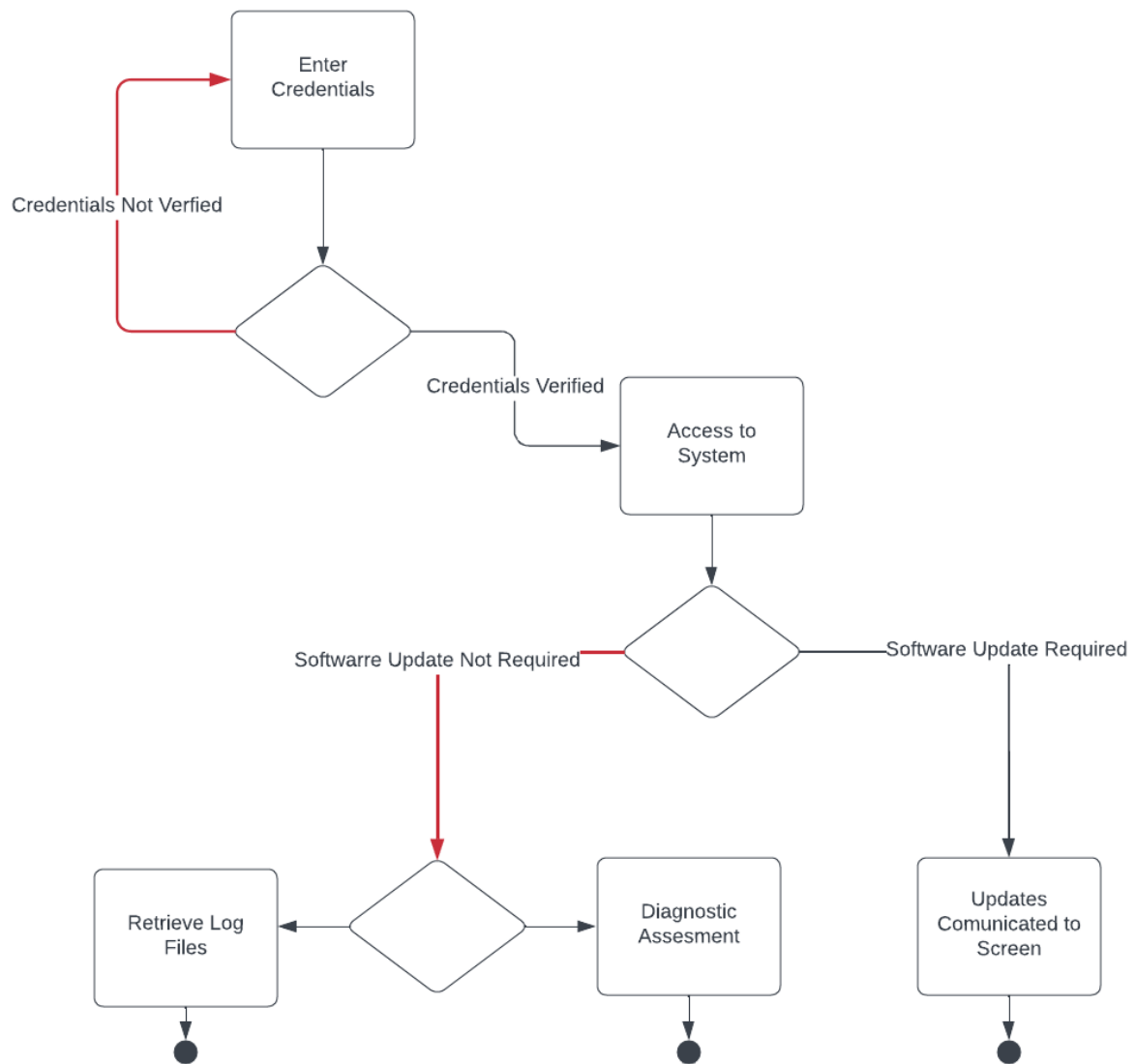
4.2 Activity Diagrams

The following five figures depict the activity diagrams for each of the aforementioned Use Cases:

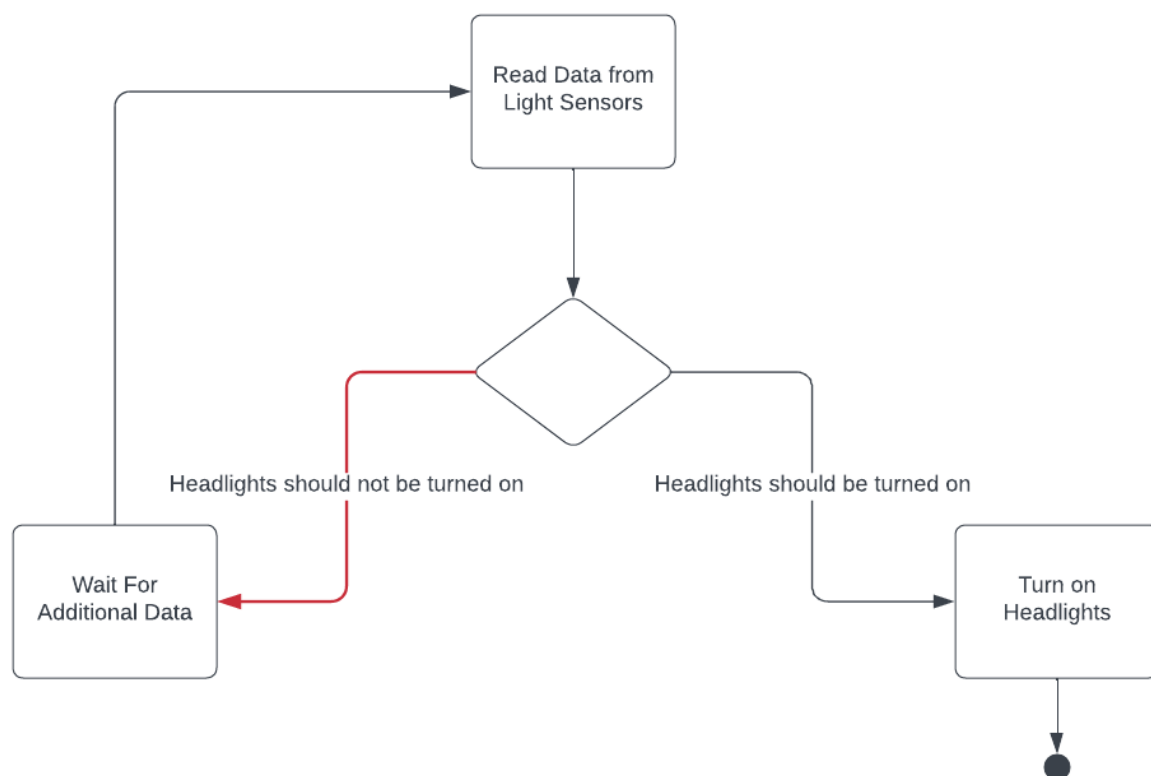
4.2.1 Activity 1 Diagram - Obstacle Detection



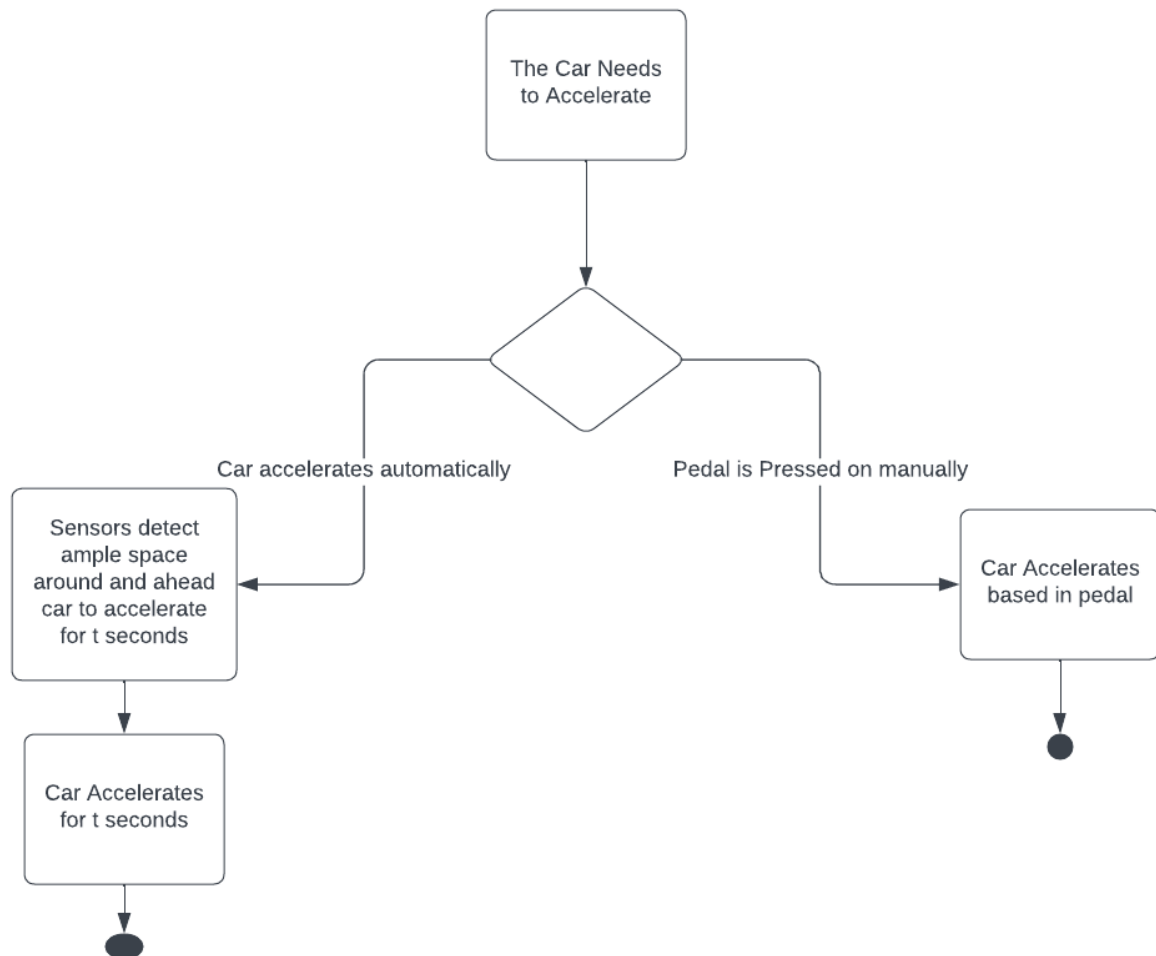
4.2.2 Activity 2 Diagram - Technician Interface



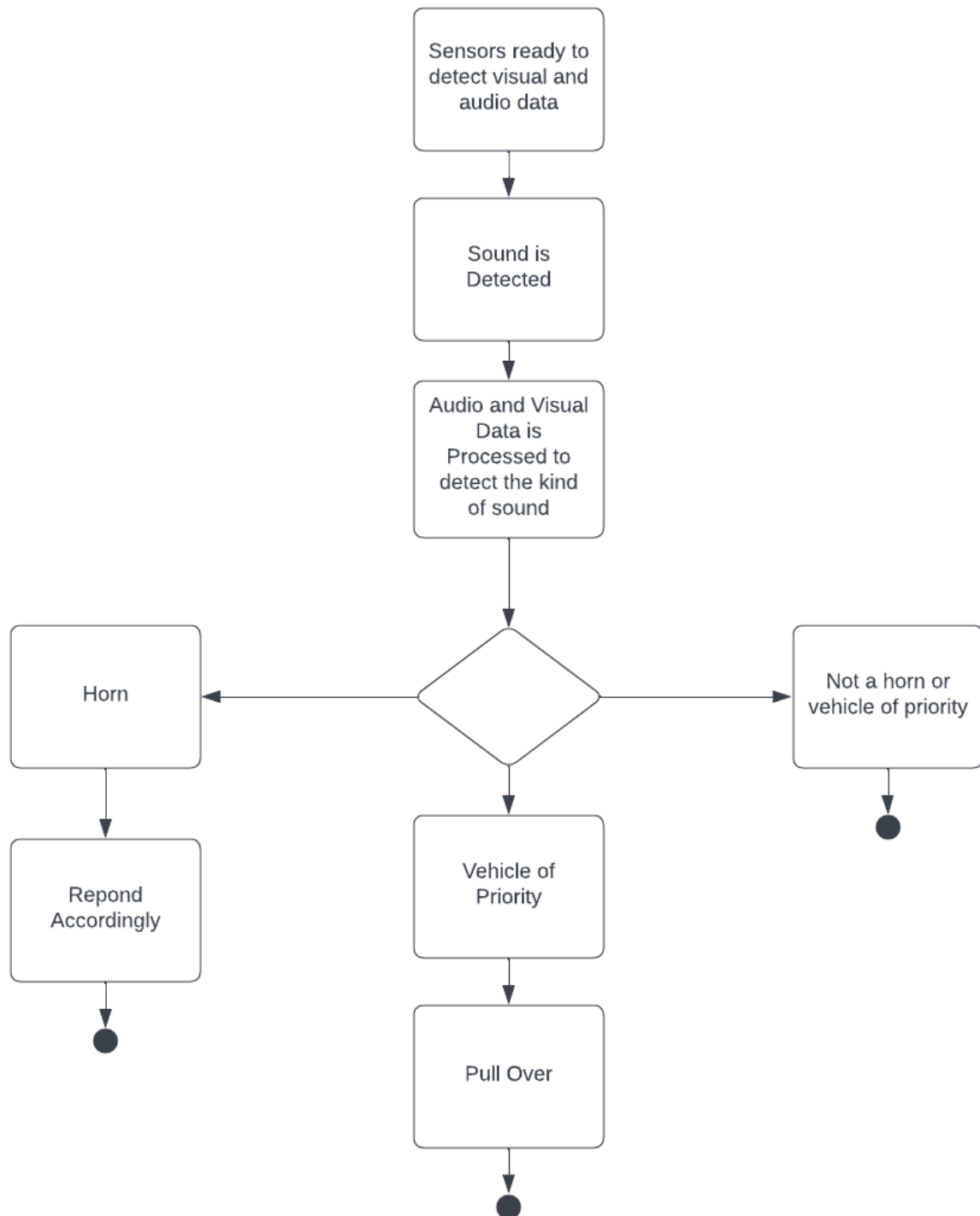
4.2.3 Activity 3 Diagram - Headlight Activation



4.2.4 Activity 4 Diagram - Vehicle Acceleration



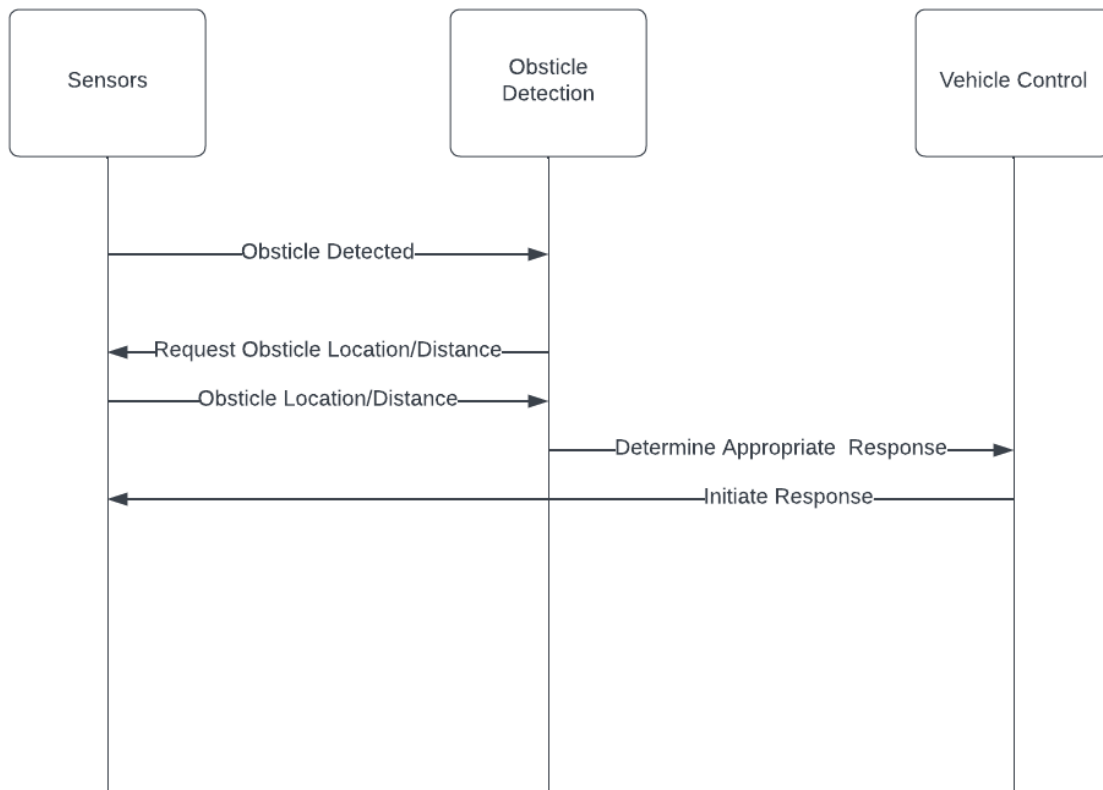
4.2.5 Activity 5 Diagram - Sound Detection



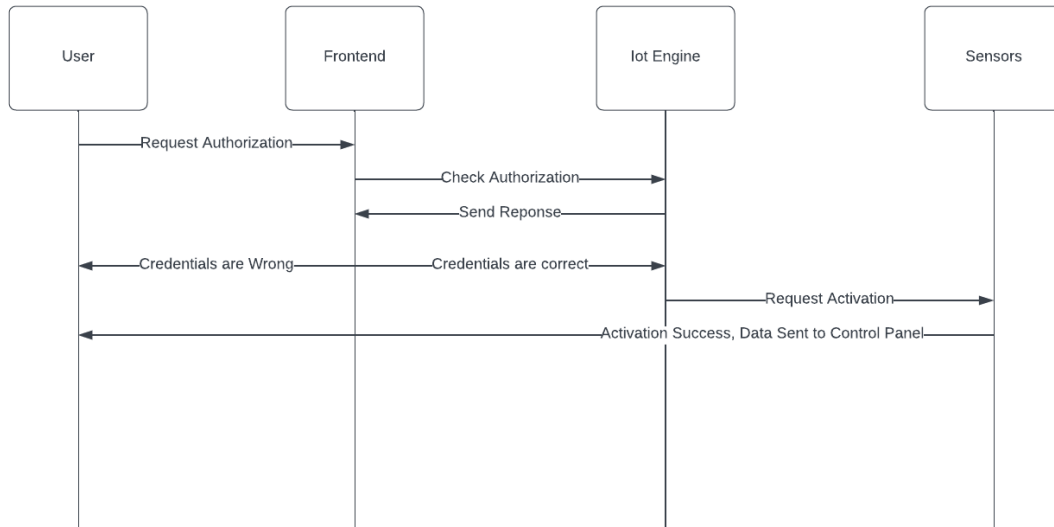
4.3 Sequence Diagrams

The following five figures depict the sequence diagrams for each of the aforementioned Use Cases:

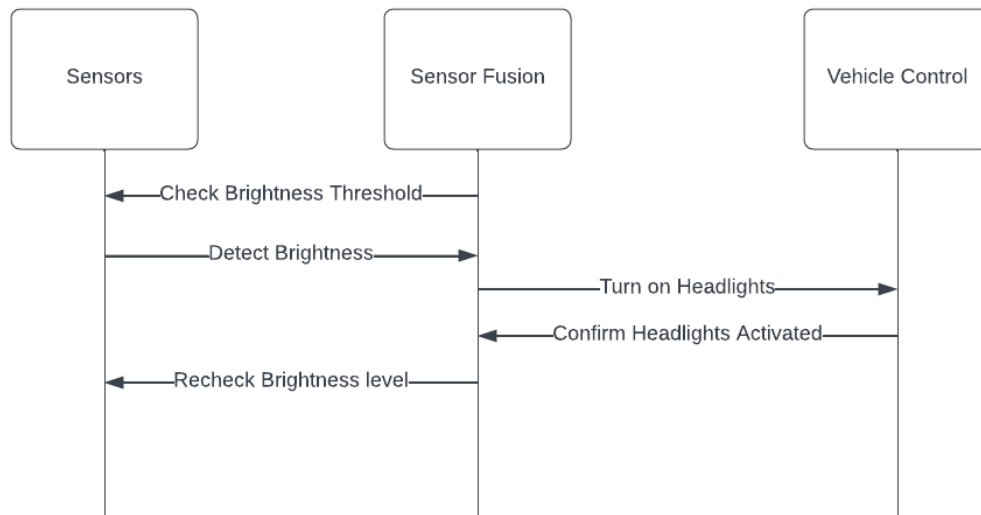
4.3.1 Sequence 1 Diagram - Obstacle Detection



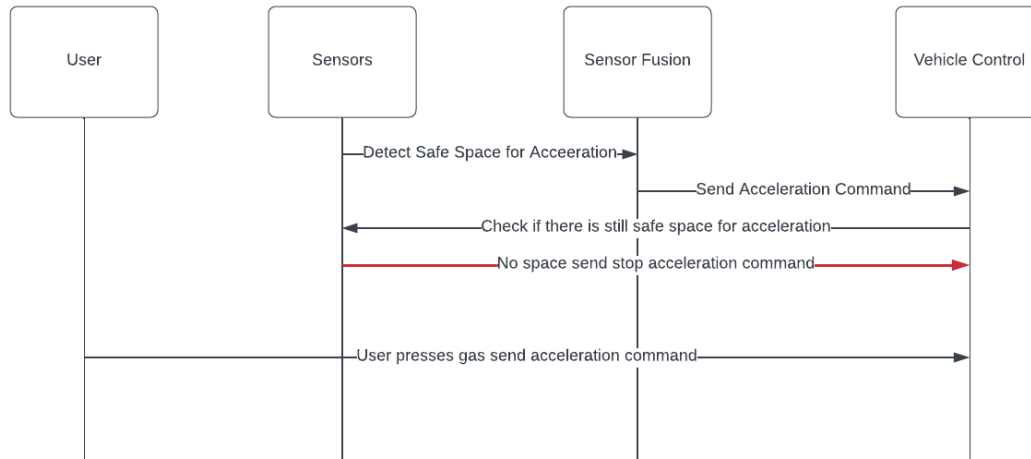
4.3.2 Sequence 2 Diagram - Technician Interface



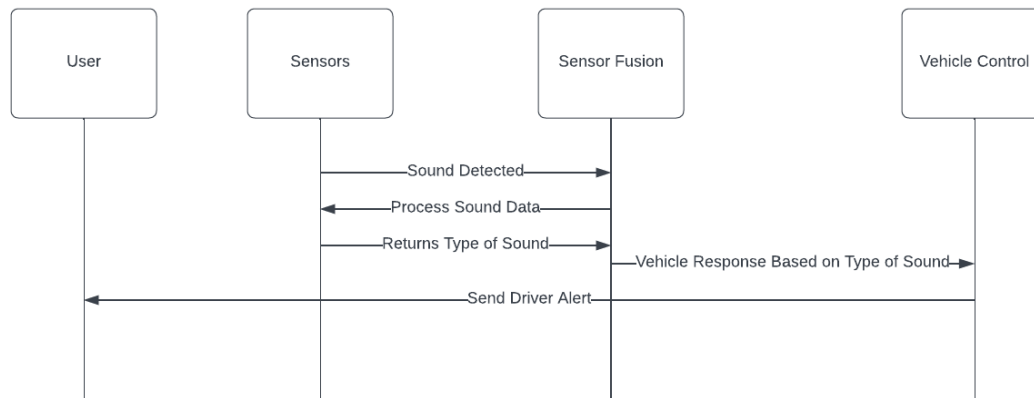
4.3.3 Sequence 3 Diagram - Headlight Activation



4.3.4 Sequence 4 Diagram - Vehicle Acceleration



4.3.5 Sequence 5 Diagram - Sound Detection



4.4 Classes

The following five class tables depict the software-based attributes and methods for each of the aforementioned Use Cases:

4.4.1 Class 1 - Obstacle Detection

Obstacle Detection
currentPosition objectPosition

currentSpeed decelerationRate
getCurrentPosition() getObjectPosition() getCurrentSpeed() decelerate() displayWarningObst()

4.4.2 Class 2 - Technician Interface

Technician Interface
username password numAttempts maxAttempts lockDuration LockTime idleDuration TimeOut
validCredentials() authenticate() lockOut() timeOut()

4.4.3 Class 3 - Headlight Activation

Headlight Activation
headlights currentTime currentBrightness sunsetTime sunriseTime
getCurrentLocation() getCurrentTime() getSunset() getSunrise() headlightsOn() headlightsOff() displayNotifLights()

4.4.4 Class 4 - Vehicle Acceleration

Vehicle Acceleration
currentPosition objectPosition accelRate accelDuration
getCurrentPosition() accelerate() accelTerminate()

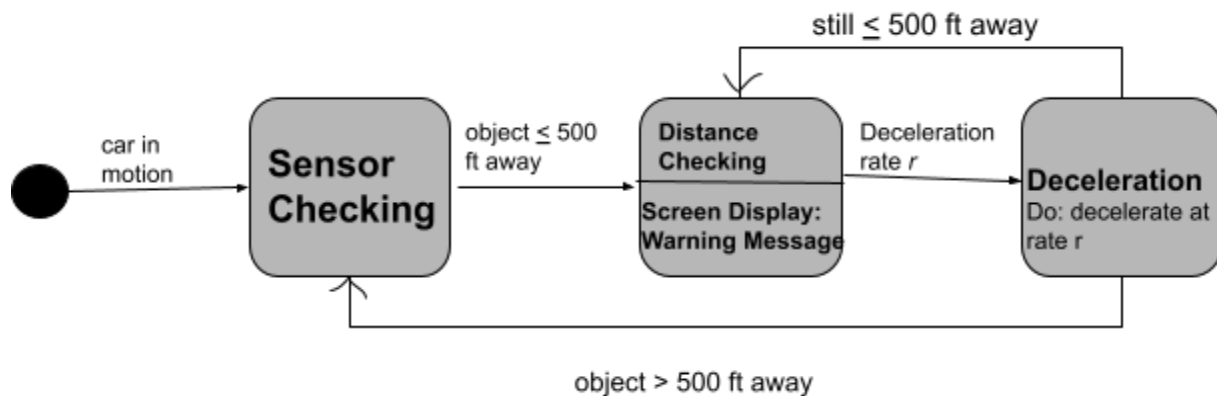
4.4.5 Class 5 - Sound Detection

Sound Detection
soundLevel soundLocation
environmentScan() displayWarningSound()

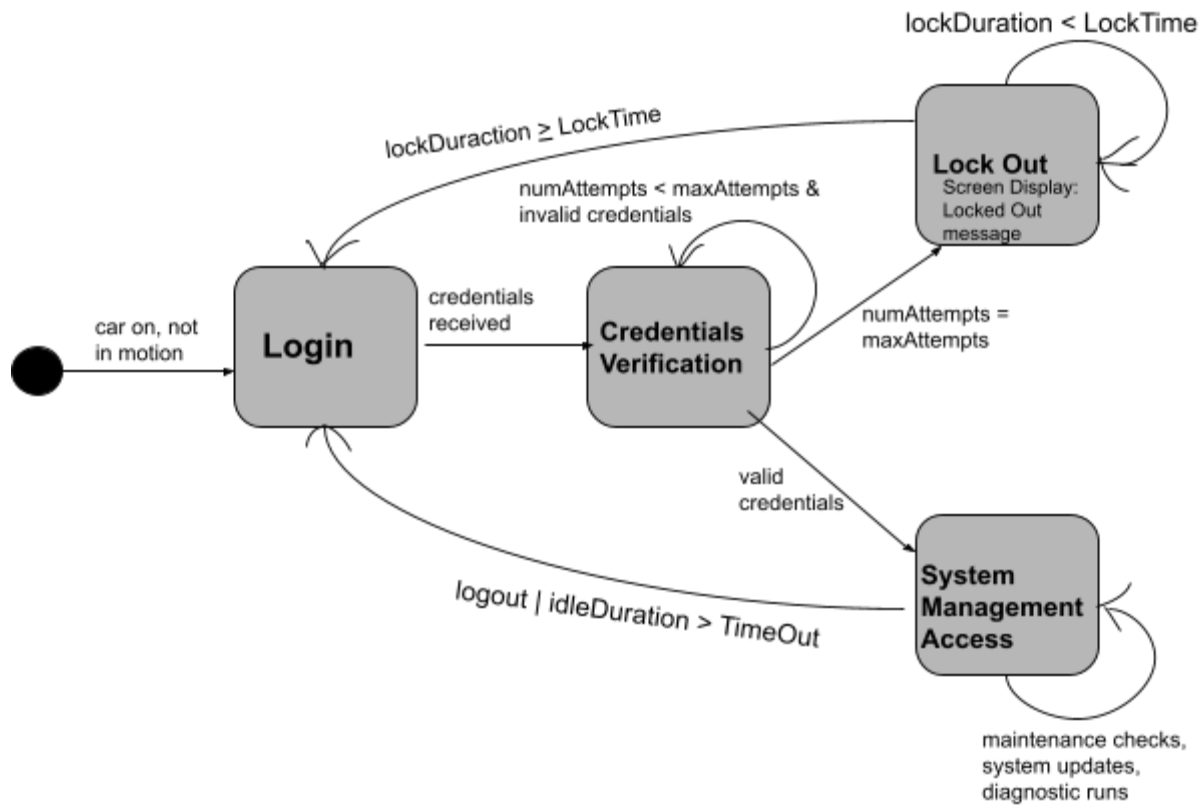
4.5 State Diagrams

The following five figures depict the state diagrams for each of the aforementioned Use Cases:

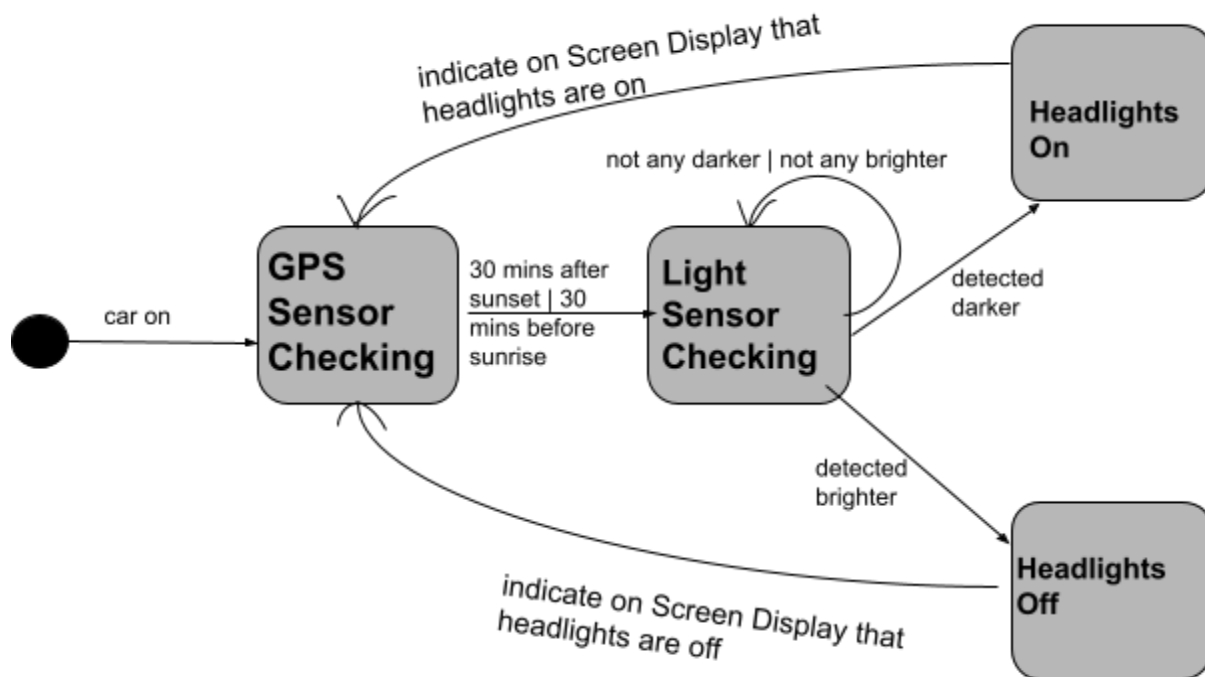
4.5.1 State Diagram 1 - Obstacle Detection



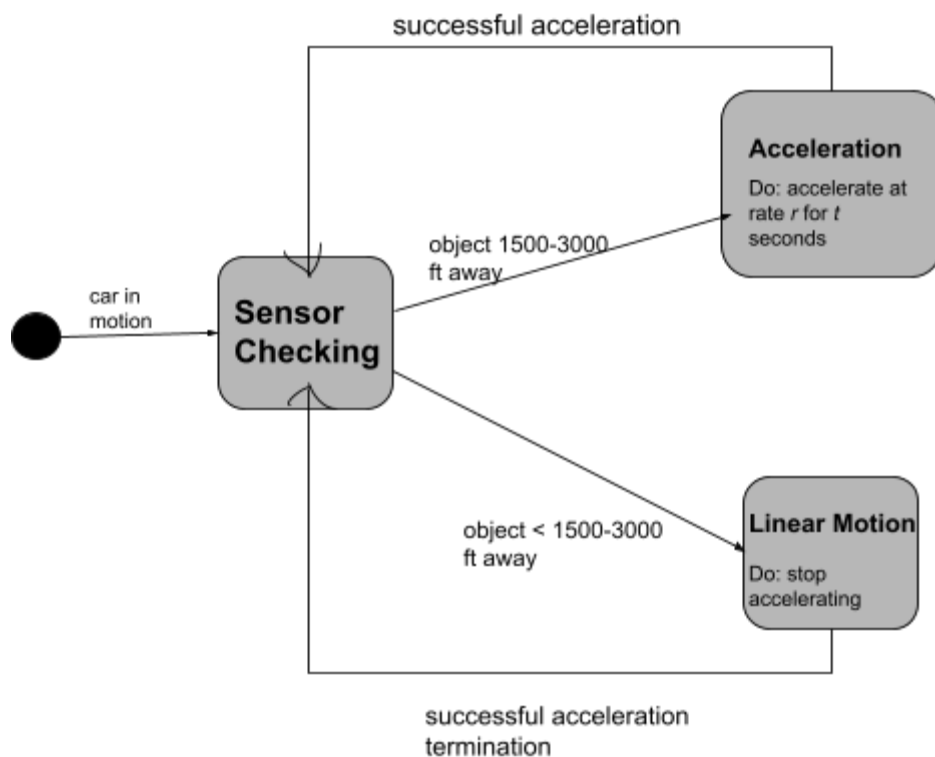
4.5.2 State Diagram 2 - Technician Interface



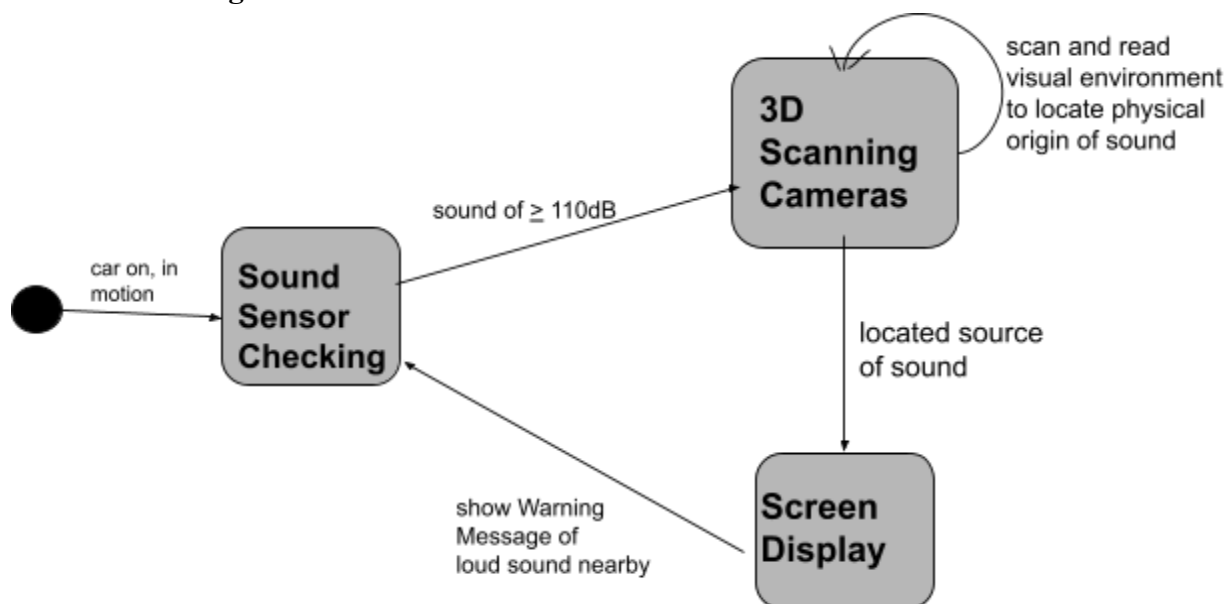
4.5.3 State Diagram 3 - Headlight Activation



4.5.4 State Diagram 4 - Vehicle Acceleration



4.5.5 State 5 Diagram 5 - Sound Detection



Section 5 - Design

5.1 Software Architecture

In this subsection, we analyze seven different architectural structures for software design. We consider them in terms of how useful they may be for this IoT HTL project then, based on these considerations, select the one we will adhere to.

5.1.1 Data-Centered Architecture

Pros:

- Every class has direct access to cloud data
- Structure is linear
 - No process should have to take any more time than another since there is only one line of communication coming from/to each software class
- Structure is not convoluted
 - There is little room for error and/or entangling of data because there is only one bidirectional connection line between each client software
 - This also makes debugging easier because there are fewer places where a bug could arise

Cons:

- Classes do not have any interlinked access
 - In order for two client softwares to communicate, they must always do so via the data store which is an extra step compared to just communicating directly between each other

5.1.2 Data Flow Architecture

Pros:

- More connectivity
 - More information can be shared at a single time between software components and interfaces
- Not just a linear flow of data
 - In some areas of this architecture, data has a cyclic flow which may better facilitate ongoing processes
- Sequential flow
 - Unlike the data-centered architecture, data flows through a series of pipes meaning that for it to have reached a certain point, it will have to have gone through previous ones too. This can make it easier to keep track of transmitted data

Cons:

- Architectural structure being more interconnected increases the difficulty of finding and resolving errors

- A bug affecting one class of software that acts as the input of another can be wrongfully propagated and develop into something whose origin is hard to detect
- Multiple incoming pipes
 - This may make it more difficult to pinpoint the origin of an error - or potentially multiple errors coming from multiple pipes

5.1.3 Call Return Architecture

Pros:

- Hierarchical structure
 - Can make data management easier
- Subprograms
 - Organizing software into programs and subprograms will make coding much easier and manageable
 - This also provides more control over smaller details in their own software sections

Cons:

- No necessary direction of data transmission
 - The connections between different programs and subprograms means that data can be transferred back and forth without necessarily adhering to a certain direction
- Potentially too many subprograms
 - The number of subprograms (and subprograms of subprograms) can quickly become exponentially larger. This may make keeping track of code difficult - especially when slight changes have to be made to each of them

5.1.4 Object-Oriented Architecture

Pros:

- Can create multiple instances of a single class
 - This means that the class attributes need to be declared only once. From then on any instances of the class will always be tied back to the class constructor
 - Can make data handling easier
- Provides reusability via polymorphism and abstraction
 - Therefore, less code can be written but more of it can be reused and/or remodeled
 - This also greatly reduces the development time

Cons:

- All software is restricted to classes and objects
 - This can make it difficult to determine which classes are needed for which functionalities

- May lead to constant revision of the number of classes and type of class attributes to best accommodate all the requirements.
- This may also become an issue in cases where only a simple bit of code is needed to do a job but needs to be implemented in a class which is just more unnecessary work/coding to do
- Not necessarily reusable in general cases
 - While object-oriented designs can be reused, they may be so specific in their architecture that they're only reusable in very specific and similar cases

5.1.5 Layered Architecture

Pros:

- Organized into specific sections of interfaces (layers)
 - This type of architectural structure can make it easier to group certain functionalities in a common layer and work on them together
- Fairly simple and cooperative framework
 - Implementing each layer does not pose a too intimidating issue
- Testing is better facilitated
 - Since each component in each level is separate from those in other layers, testing can be done with more ease. The software engineers can rest assured that they are definitely only testing the components of that specific layer without interference of any other layers' components.

Cons:

- Potential harmful effects of layers on each other
 - In the event where a certain later is dependent on another, if one layer's software is not functioning, it may cause a chain reaction of bugs down the rest of the dependencies
- Revisions to one layer may force revisions to them all
 - In some cases, one change made to one component in one layer may be enough to force the software engineers to revise the code in all other layers

5.1.6 Model View Controller Architecture

Pros:

- Parallel development can be done
 - The three subsets of this architecture - model, view, and controller - can be worked on simultaneously. Then, when finished, links will be made between all three
 - This can make the development process much faster
- Software revisions will not necessarily affect the entire architecture
 - If a modification must be made to the code in one part of this model, it can be handled only inside that part. As long as the links between the three of

them are unharmed, the change in code should not force the entire structure to be remodeled

Cons:

- Parallel development may also be a detriment
 - Developing all three parts of this architecture at once may hinder communication between them until time to link them together.
 - This means that there may be times when one part accounts for or implements something that another part may not recognize or may even reject
- Structuring each part of this architecture doesn't have to follow the same model
 - The way the controller part is constructed does not necessarily have to mimic that of the model or the view parts.
 - This can lead to code that potentially seems disorganized and disconnected between the three

5.1.7 Finite State Machine Architecture

Pros:

- Transitions between states
 - For a state to transition to another, it is definitely known what condition was met.
 - This can make managing data transmission much easier because it can be more restricted to follow a certain path without getting lost or misplaced along the way
- Increased predictability can be reassuring
 - When a path for data to flow through is essentially mapped out via states and their transitions, it can be easier to predict how data will be transmitted once it's available
 - Therefore, if something goes wrong, it may be easier to find exactly where it happened and why it did based on the current state and the transitions from it
- Only one state active at a time
 - Data will only ever be at one state at any given time before moving along the next transition. Therefore, there is less to worry about when it comes to tracking it and making sure that it is where it should be

Cons:

- Multiple transitions to/from states
 - While state transitioning is not a detrimental factor, it can quickly become difficult to keep track of all the potential paths data may flow through
- May be difficult to implement on larger scales
 - While incredibly effective on smaller scales, finite state machines can become increasingly difficult to implement in bigger projects.

- There are more possibilities to account for and, therefore, more transitions to define. With more work to be done, there is more potential for making a mistake somewhere and having difficulty finding it
-

After evaluating all of these different types of architecture, we shall be adhering to the Call Return Architecture. This will allow us to have more control over individual subsections and subprograms.

5.2 Interface Design

5.2.1 Cruise Control

- The cruise control is activated when the driver presses the button. The car receives data from its sensors to ensure that it follows its speed limits and adjusts to traffic laws and conditions.
 - **5.2.1.1 Variables:**
 - vehicleSpeed(double)
 - vehicleCoordinates(tuple)
 - vehicleSpeed(double)
 - cruiseControlPressed(bool)
 - **5.2.1.2 Functions:**
 - getSpeed
 - getLocation
 - setSpeed
 - checkDrivingConditions
 - displayToUser,

5.2.2 Software Updates

- The driver is prompted to update their car software. If they choose to update, the software begins and installs updates. If they choose not to update, they'll be redirected to choose a time to initiate the update.
 - **5.2.2.1 Variables:**
 - checkUpdatedToDate(bool)
 - **5.2.2.2 Functions:**
 - update
 - delaySoftwareUpdate
 - displayToUser

5.2.3 Login/Sign up

- The driver is prompted to authenticate themselves via pin, fingerprints, etc. If their authentication matches with the data stored in the car, grant them access to the car. Else if the driver is a new driver, prompt them to sign up. Otherwise do not initiate anything.
 - **5.2.2.1 Variables:**
 - userName(string)
 - address(string)
 - phoneNumber(string)
 - driverLicenseID(string)
 - dateOfBirth (date)
 - **5.2.2.2 Functions**
 - Login
 - Signup

5.2.4 Parking

- The automatic parking feature is activated when the driver presses the virtual button. The vehicle collects data from cameras and GPS and creates a path for the vehicle to follow. If the gear is parked, the vehicle takes control of steering and acceleration. The path is followed and control is passed back to the driver.
 - **5.2.4.1 Variables:**
 - vehicleSpeed(double)
 - vehicleCoordinates(tuple)
 - vehicleSpeed(double)
 - cruiseControlPressed(bool)

5.3 Component-Level Design

5.3.1 Client Software/ Operating System

- The client software operating system is to allow the overall communication between the components of the car and the features of each. It must be noted that the variables include the variables of all functions of the car.

- **5.3.1 Variables:**

- isOn(bool)

5.3.2 Sensor Fusion Localization

- The vehicle needs the components for localization, which consists of Radio Wave Detectors, a GPS, an IMU, and Sonar Detectors. Together, these components allow the vehicle to gain a sense of surroundings and directions to the vehicles.

- **5.3.2.1 Variables:**

- nearCollision(double)
 - getGPSCoordinates()
 - getSoundDistance()

- **5.3.2.2 Functions:**

- Spatial Awareness Through Distance

5.3.3 Sensor Fusion Perception

- The vehicle needs the components for perception, which consists of the cameras and laser scanners. Together, these components allow the vehicle to identify the objects in the surroundings

- **5.3.3.1 Variables:**

- objectDistance()
 - identifyObject(object)
 - objectVelocity(double, object)

- **5.3.3.2 Functions:**

- Identify and Distinguish the Surrounding Objects

5.3.4 Vehicle Control

- The vehicle needs the components for vehicle control to work with all features, which consists of steering, braking, and acceleration. Together, these components allow the vehicle to have movement control.

- **5.3.4.1 Variables:**

- changeAcceration(double)
 - changeVelocity(double)
 - stop()
 - changeDirection(double *angle change*)

- **5.3.4.2 Functions:**
 - Allow movement control of the vehicle.